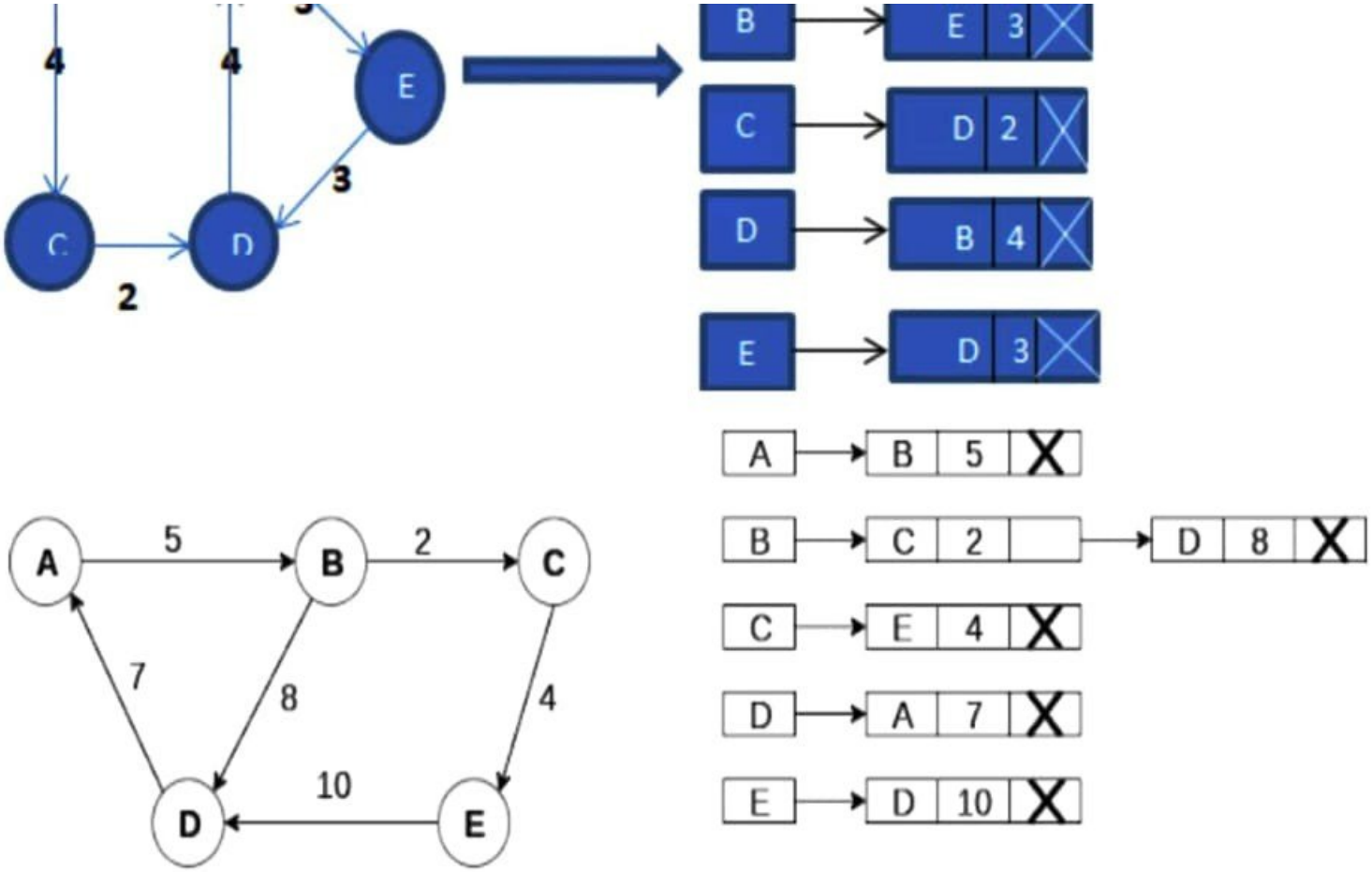


Fig. Adjacency List representation of Weighted Directed Graph/Directed Weighted Graph

Pros: Space-efficient for sparse graphs as it consumes only **O(V + E)** space. Adding a vertex takes **O(1)** time (amortized, if using dynamic arrays or hash maps). Iterating over all edges of a vertex is efficient—only takes time proportional to the vertex's degree**.** More memory-efficient for large, sparse graphs compared to adjacency matrix.

Cons: Checking if there is an edge from vertex 'u' to vertex 'v' is **O(degree of u)** in worst case**.** Removing an edge can take **O(degree of u)** time due to the need to search the list. Slightly more complex to implement compared to adjacency matrix.

Algorithm:

Algorithm for Graph creation using Adjacency Matrix:

1. Declare an array of M[size][size] which will store the graph.
2. Enter how many nodes you want in a graph.
3. Enter the edges of the graph by two vertices each, say V^i , V^j indicates some edge
   i. If the graph is directed set M[i][j]=1.
   ii. If graph is undirected set M[i][j] =1 and M[j][i] =1.
4. When all the edges of the desired graph is entered print the graph M[i][j].

Algorithm for Graph creation using Adjacency List:

1. Declare node structure for creating adjacency list.
2. Initialize an array of nodes. This array will act as head nodes. The index of head[ ] will be the starting vertex.
3. The create function will create the adjacency list.

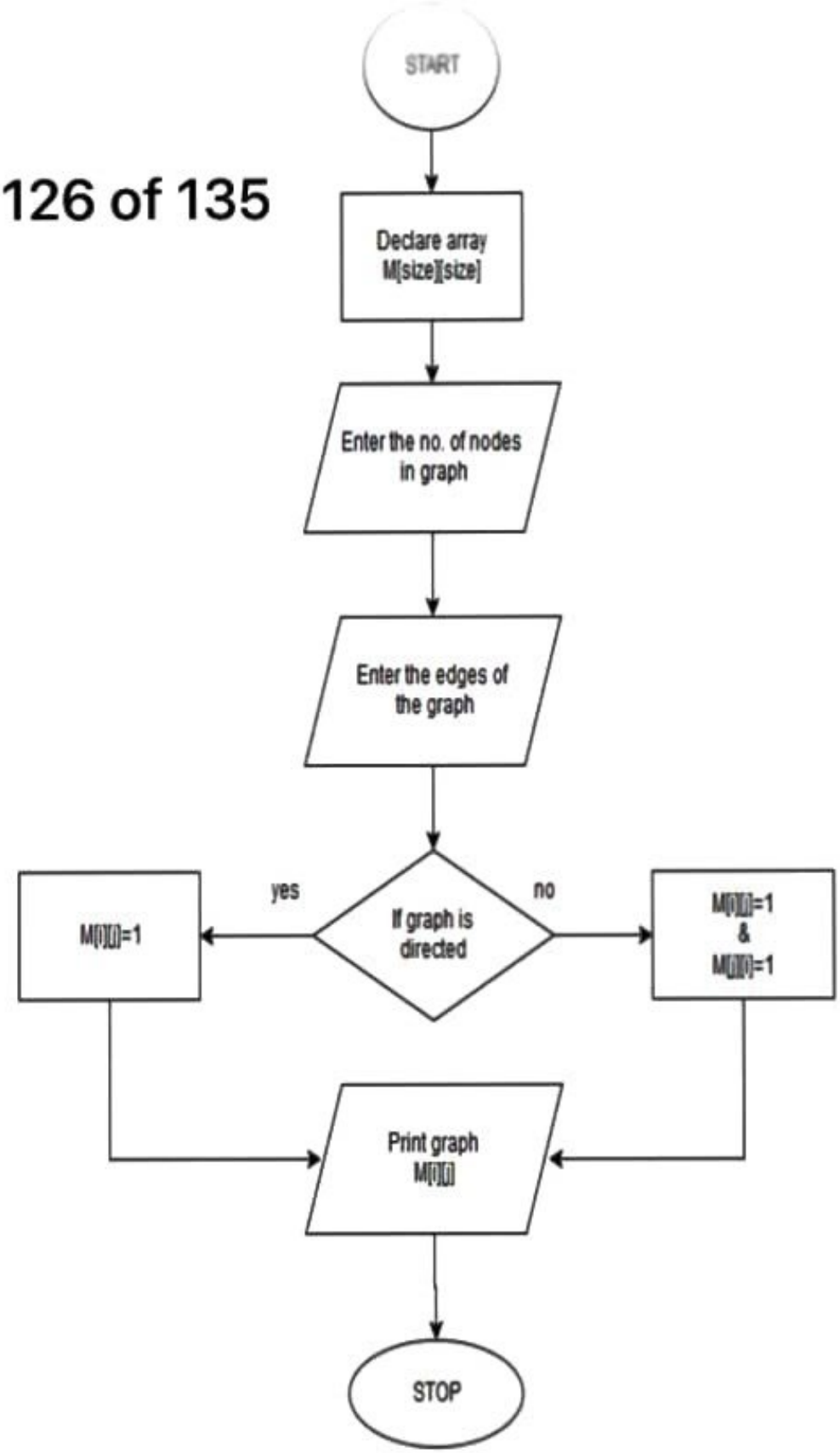


Fig. Flowchart for representation of Graph

Input: A Weighted Graph as a representation of flight paths between cities.
Output: Results based on the implemented flight paths between cities as a weighted graph's adjacency list or adjacency matrix and checked whether the graph is connected or not.

Conclusion: Concept of graph, types of graph and it's representations were studied, understood and implemented on flight paths between cities successfully.

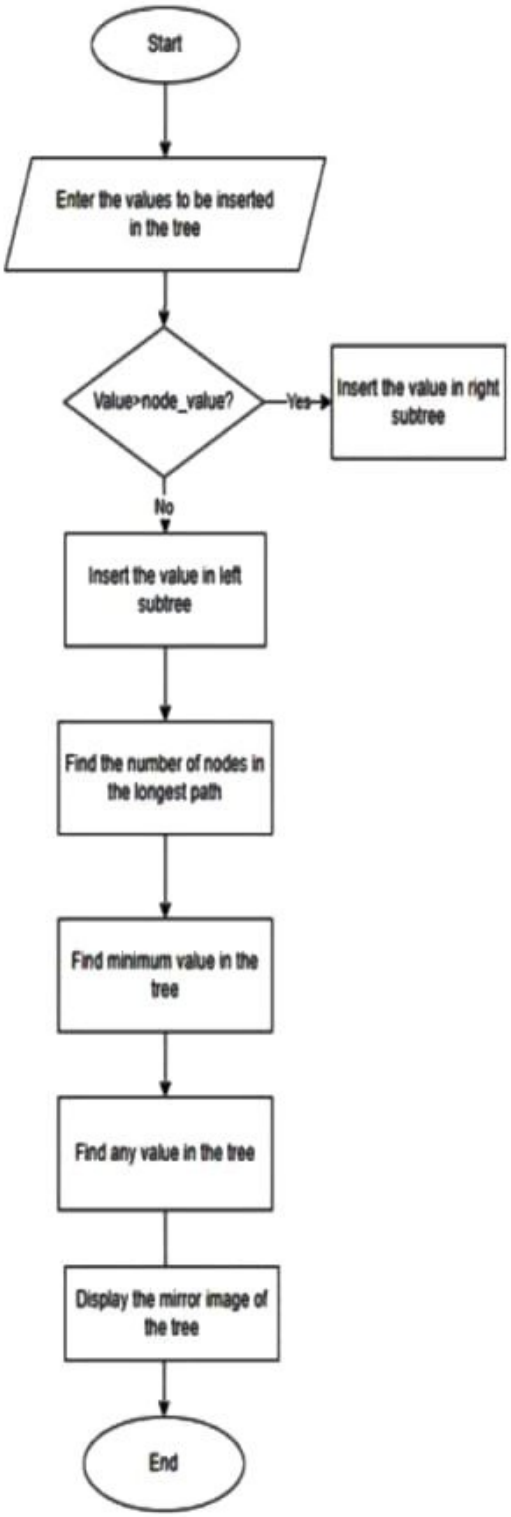


Fig. Flowchart for Implementation of Binary Search Tree

Input: An empty Binary Search Tree.
Output: Results based on formation of binary search tree structure with its basic operations like insertion, searching and Traversal in tree.

Conclusion: Concept of tree, binary tree, binary search tree, it's basic operations like insertion, searching, traversal, swapping of left and right pointers and it's implementations were studied, understood and implemented to construct binary search tree successfully.

OR

Practical No. 8

Title: Implementing Binary Search Tree and it's operations.
Problem Statement:
A Dictionary stores keywords and its meanings. Provide facility for adding new keywords, deleting keywords,

Algorithm:

Algorithm to insert a node in the binary tree:

1. Create a new BST node and assign values to it.
2. insert(node, key)
   i) if root == NULL,
   return the new node to the calling function.
   ii) if root=>data < key
   call the insert function with root=>right and assign the return value in root=>right.
   root->right = insert(root=>right,key)
   iii) if root=>data > key
3. call the insert function with root->left and assign the return value in root=>left.
4. root=>left = insert(root=>left,key)
5. Finally, return the original root pointer to the calling function.

Algorithm to find the minimum value in binary tree:

1. Define Node class which has three attributes namely: data, left and right. Here, left represents the left child of the node and right represents the right child of the node.
2. When a node is created, data will pass to the data attribute of node and both left and right will be set to null.
3. Define another class which has an attribute root.

Practical No. 9

Title: Implementing Graph and it's representations.

Problem Statement:

There are flight paths between cities. If there is a flight between city A and city B then there is an edge between the cities. The cost of the edge can be the time that flight takes to reach city B from A or the amount of fuel used for the journey. Represent this as a graph. The node can be represented by the airport name or name of the city. Use adjacency list representation of the graph or use adjacency matrix representation of the graph. Check whether the graph is connected or not.

Aim:

Write C/C++ program to represent flight paths between cities with cost of the edges as a graph. After representing the graph -

- Use adjacency list representation of the graph.
- Use adjacency matrix representation of the graph.
- Check whether the graph is connected or not.

Objectives:

- To study and understand the basic concept of Non-Linear Data Structure.
- To study and understand graph, types of graph and it's representations.
- To implement flight paths between cities with cost of the edges as a graph.
- To implement adjacency list and matrix representations and show whether they are connected or not.

Prerequisites:

- Knowledge of C and C++ Programming, Object Oriented Programming.
- Knowledge of Arrays and Linked Lists.

Outcomes:

- Understand the Graph data structure.
- Implement flight paths between cities with cost of the edges as a graph.
- Represent Graph's adjacency list and adjacency matrix.
- Check Graph is a Connected Graph or not.
- Solve real world problem logically using Graph Data Structure.

Software & Hardware requirements:

- Open Source C/C++ Programming tools like G++/GCC or Eclipse.
- 64-bit Open source Linux or its derivative.

Theory Concept:

Graph

A graph is a non-linear data structure consisting of nodes or vertices which are connected to other nodes through edges or links. Nodes are circles represented by numbers or alphabets. They store the data. Two nodes are connected by a line called Edge. Edge can be directed or undirected. Basically, pairs of vertices are called edges. A graph is formally defined as G = (V, E), where V is the set of vertices and E is the set of edges.

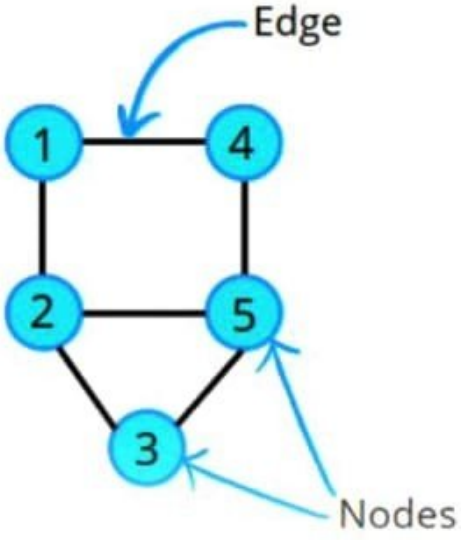


Fig. Graph Data Structure

In the above example, the edge can go from 1 to 4 or from 4 to 1, i.e. a bidirectional edge can be in both directions, hence called an **undirected edge**. Thus, the pairs (1,4) and (4,1) represent the same edge.

Practical No. 8

Title: Implementing Binary Search Tree and it's operations.

Problem Statement:

Beginning with an empty binary search tree, construct a binary search tree by inserting the values in the order given. After constructing a binary tree -

i. Insert new node
ii. Find number of nodes in longest path from root
iii. Minimum data value found in the tree
iv. Change a tree so that the roles of the left and right pointers are swapped at every node
v. Search a value.

Aim:

Write C/C++ program to construct a binary search tree from an empty binary search tree by inserting the values in the order given. After constructing a binary tree -

- Insert new node
- Find number of nodes in longest path from root
- Minimum data value found in the tree
- Change a tree so that the roles of the left and right pointers are swapped at every node.
- Search a value.

Objectives:

- To study and understand the basic concept of Non-Linear Data Structure.
- To study and understand tree, binary tree, binary search tree and it's basic operation in Data structure.
- To implement Binary Search Tree and perform various operations on it.
- To swap the left and right pointers at every node in binary search tree.

Prerequisites:

- Knowledge of C and C++ Programming, Object Oriented Programming.
- Knowledge of Arrays, Linked Lists, Stack and Queue.

Outcomes:

- Classify the Tree data structures.
- Implement Binary Search Tree to store a number in it.
- Perform basic Tree Operations like Insertion, Searching and Traversal.
- Solve real world problem logically using Tree Data Structure.

Software & Hardware requirements:

- Open Source C/C++ Programming tools like G++/GCC or Eclipse.
- 64-bit Open source Linux or its derivative.

Theory Concept:

Tree

A tree is a hierarchical, non-linear data structure composed of nodes connected by edges. It represents data with a parent-child relationship, where each node can have multiple child nodes, except for the root, which has no parent.

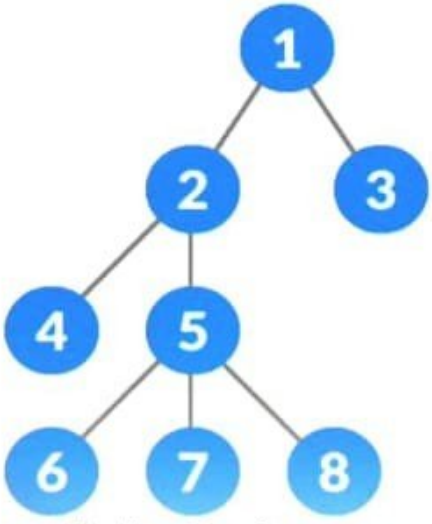


Fig. Tree Data Structure






Basic Terminologies of Tree

- Nodes: A node is an entity that contains a key or value and pointers to its child nodes. The last

Algorithm for display() function

display() - Displays the elements of a Queue.

Step 1: Check whether queue is EMPTY. (front == rear)
Step 2: If it is EMPTY, then display "Queue is EMPTY!!!" and terminate the function.
Step 3: If it is NOT EMPTY, then define an integer variable 'i' and set 'i = front+1'.
Step 4: Display 'queue[i]' value and increment 'i' value by one (i++). Repeat the same until 'i' value is equal to rear (i <= rear)

Flowchart:

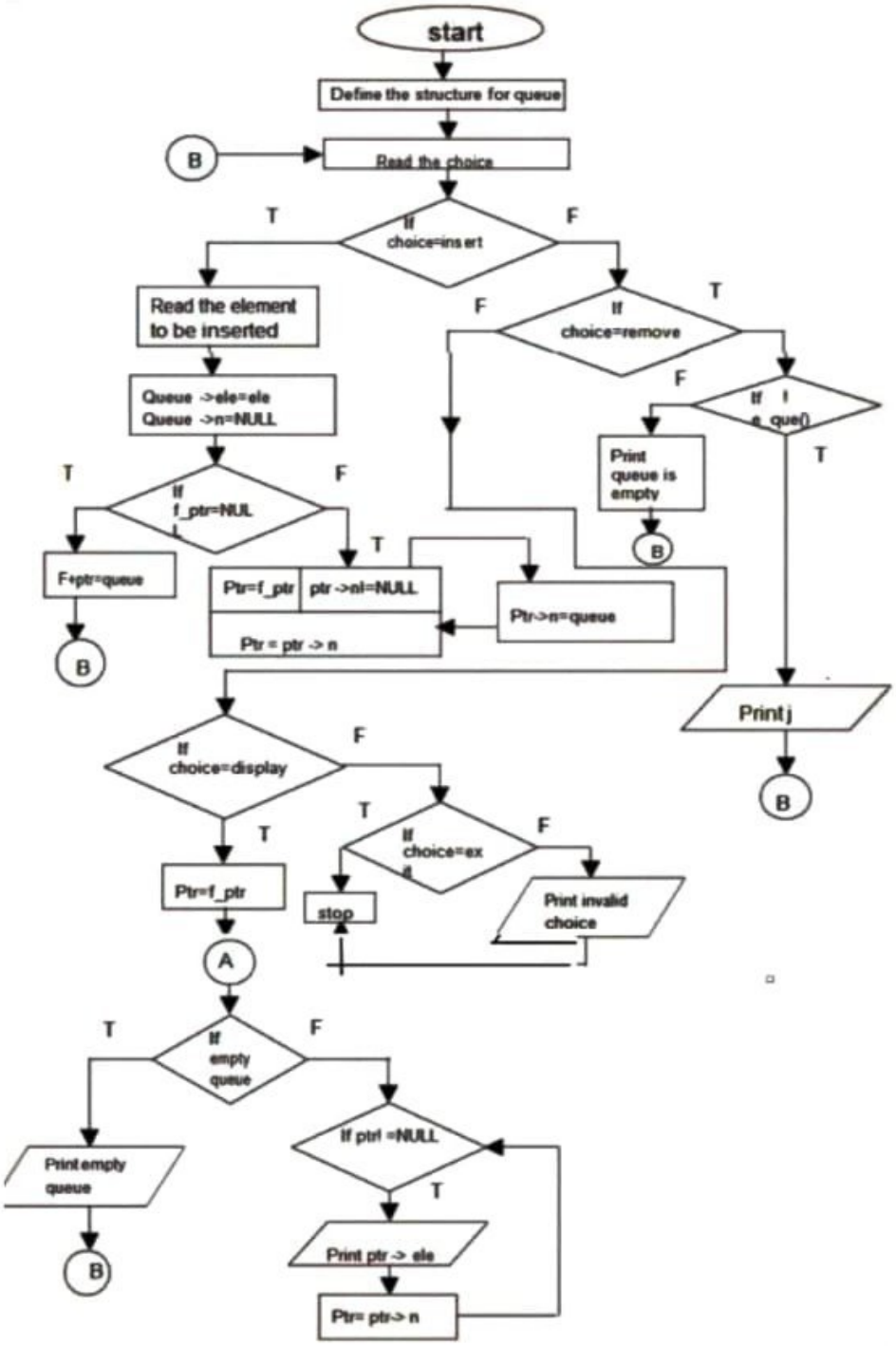


Fig. Flowchart for Job Queue Implementation of Pizza Ordering

Input: Size of array, Orders as Elements in queue, order details like Order _ID, Customer_Name.
Output: Results based on addition and deletion(modification) of jobs/orders in job queue.

Conclusion: Concept of Queue, it's operations and implementations, Real World Job Queue using Linear Queue were studied, understood and implemented to simulate the system using job queue using array successfully.

Practical No. 7

Title: Implement Linear Queue as Real World Job Queue & its operations.

Problem Statement:

Pizza parlor accepting maximum M orders. Orders are served in first come first served basis. Order once placed cannot be cancelled. Write a program to simulate the system using job queue using array. Write functions to add jobs and delete jobs from the queue.

Aim:

Write C/C++ program to simulate the Pizza ordering system using job queue using array of Pizza parlor accepting maximum M orders served in first come first served basis which cannot be cancelled once placed using job queue using array. Write functions to add jobs and delete jobs from the queue.

Objectives:

- To study Queue Data Structure.
- To study & implement Job Queue.
- To perform Job Queue for Pizza ordering using array.
- To display available orders/jobs, book/add and cancel/delete (modify) jobs/orders.

Prerequisites:

- Knowledge of C and C++ Programming, Knowledge of pizza ordering, Object Oriented Programming, Array and Linked List.

Outcomes:

- Implement Queue and perform various operations on it.
- Solve real world problem logically using Queue Data Structure.

Software & Hardware requirements:

- Open Source C/C++ Programming tools like G++/GCC or Eclipse.
- 64-bit Open source Linux or its derivative.

Theory Concept:

Queue

A queue is a linear data structure where elements are stored in the FIFO (First In First Out) manner where the first element inserted would be the first element to be deleted. Insertion happens from rear end only and Deletion happens from front end only.

